

HIGH-LEVEL DESIGN DOCUMENT

Personal Portfolio Website

Full-Stack Web Application — FastAPI · PostgreSQL · AI-Powered Roadmap

Document Type	High-Level Design (HLD)
Current Phase	Phase 0 — Planning & System Design (In Progress)
Core Stack	FastAPI · PostgreSQL · SQLAlchemy · Vanilla JS

1

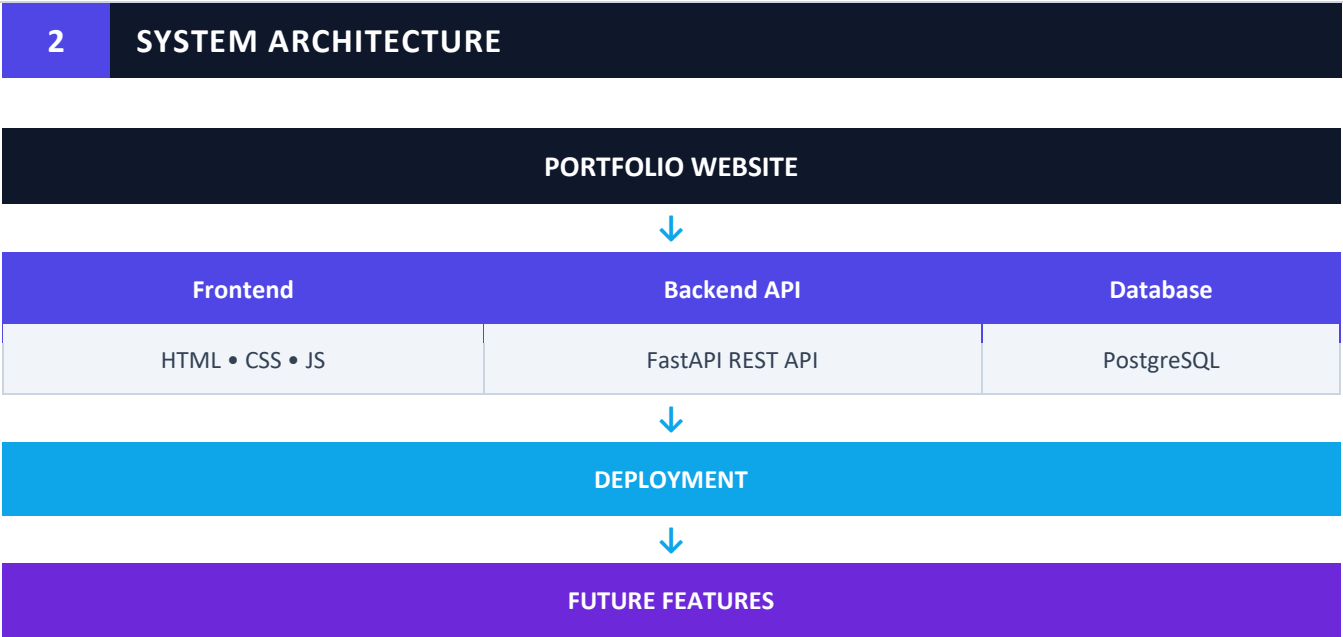
PROJECT OVERVIEW

► 1.1 Objective

The objective of this project is to design and develop a modern, responsive, and scalable personal portfolio website that showcases my technical skills, projects, experience, certifications, blogs, and professional journey.

Unlike a traditional static portfolio, this application will evolve into a full-stack web application powered by FastAPI and PostgreSQL. The long-term vision is to transform it into an AI-powered portfolio capable of answering questions about my work, projects, and experience through an intelligent chatbot.

The project is divided into multiple development phases, allowing each component to be designed, implemented, tested, and deployed independently.



3

TECHNOLOGY STACK

LAYER	TECHNOLOGY
Frontend	HTML5, CSS3, Vanilla JavaScript
Backend	FastAPI
ORM	SQLAlchemy
Database	PostgreSQL
Validation	Pydantic
API Documentation	Swagger / OpenAPI
Version Control	Git & GitHub
Deployment	Vercel + Render + Neon PostgreSQL
Future AI	OpenAI/Gemini + pgvector

4

DEVELOPMENT ROADMAP

The project is divided into six major phases.

PHASE 0 — Planning & System Design**IN PROGRESS****Objective**

Design the overall architecture before implementation.

Deliverables

- Functional Requirements
- Non-Functional Requirements
- High-Level Design
- Low-Level Design
- Folder Structure
- Database Design
- API Contracts
- Deployment Strategy
- Future Roadmap

PHASE 1 — Frontend Development**COMPLETED****Objective**

Develop a responsive, modern, and interactive user interface.

Technologies Used

- HTML5
- CSS3
- Vanilla JavaScript

Website Sections

- Navigation
- Hero Section
- About Me
- Experience
- Projects
- Skills
- Current Focus
- Education
- Certifications
- Blog

- Contact
- Footer

Features Implemented — User Interface

- Responsive Layout
- Glassmorphism Design
- Dark & Light Theme
- Mobile Navigation
- Sticky Navigation Bar

Features Implemented — User Experience

- Scroll Progress Indicator
- Back-to-Top Button
- Smooth Scrolling
- Reveal Animations
- Typing Animation
- Animated Skill Bars
- Counter Animations

Features Implemented — Projects

- Live Search
- Category Filters
- Dynamic Filtering
- Empty State

Features Implemented — Contact

- Client-side Validation
- Loading State
- Toast Notifications

Current Status

✓ Frontend development has been completed.

PHASE 2 — Database Design**PLANNED****Objective**

Convert all static website content into structured and manageable database records.

Database

portfolio_db

Tables

Users — Stores administrator information.

Projects — Stores all portfolio projects.

Blogs — Stores blog articles.

Skills — Stores technical skills and proficiency levels.

Experience — Stores work experience.

Education — Stores educational qualifications.

Certifications — Stores professional certifications.

Contact Messages — Stores visitor contact submissions.

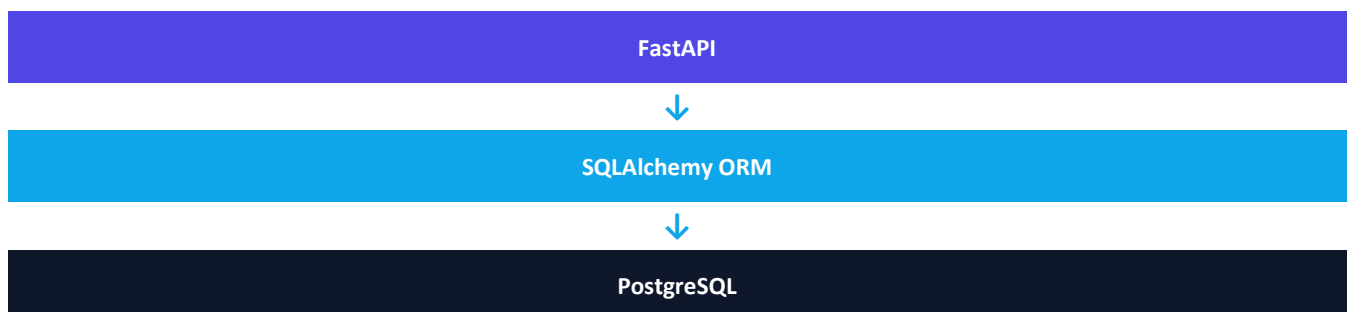
Visitor Analytics — Stores visitor statistics for future analytics.

Deliverables

- Entity Relationship Diagram (ERD)
- PostgreSQL Database
- SQL Scripts
- Constraints
- Indexes
- Sample Data

PHASE 3 — Backend Development**PLANNED****Objective**

Develop a scalable REST API using FastAPI.

Technology Stack**Backend Folder Structure**

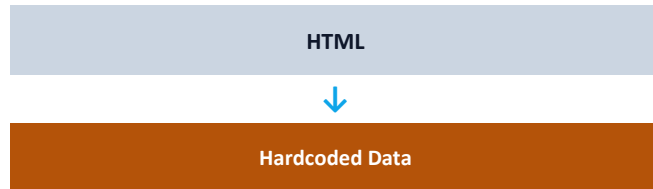
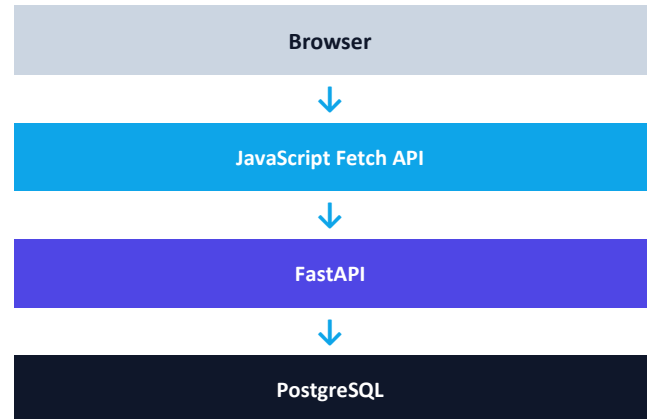
```
backend/  
├── app/  
│   ├── api/  
│   ├── core/  
│   ├── database/  
│   ├── models/  
│   ├── schemas/  
│   ├── services/  
│   ├── utils/  
│   └── main.py  
├── requirements.txt  
└── README.md
```

Backend Features

- CRUD Operations
- Pydantic Validation
- Logging
- Exception Handling
- Pagination
- Filtering
- Swagger Documentation
- Database Migrations
- Environment Configuration

PHASE 4 — API Integration**PLANNED****Objective**

Replace all hardcoded frontend data with API-driven content.

Current Architecture**Target Architecture****Dynamic Sections**

The following sections will retrieve their data directly from the backend:

- Projects
- Skills
- Experience
- Education
- Certifications
- Blogs
- Contact Form

Example**Current Implementation**

```
const projects = [  
  ...  
];
```

Future Implementation

```
const response = await fetch("/api/projects");  
const projects = await response.json();
```

PHASE 5 — Deployment	PLANNED
----------------------	---------

Objective

Deploy the complete application for public access.

Frontend Hosting

- Vercel (Recommended)
- Netlify
- GitHub Pages

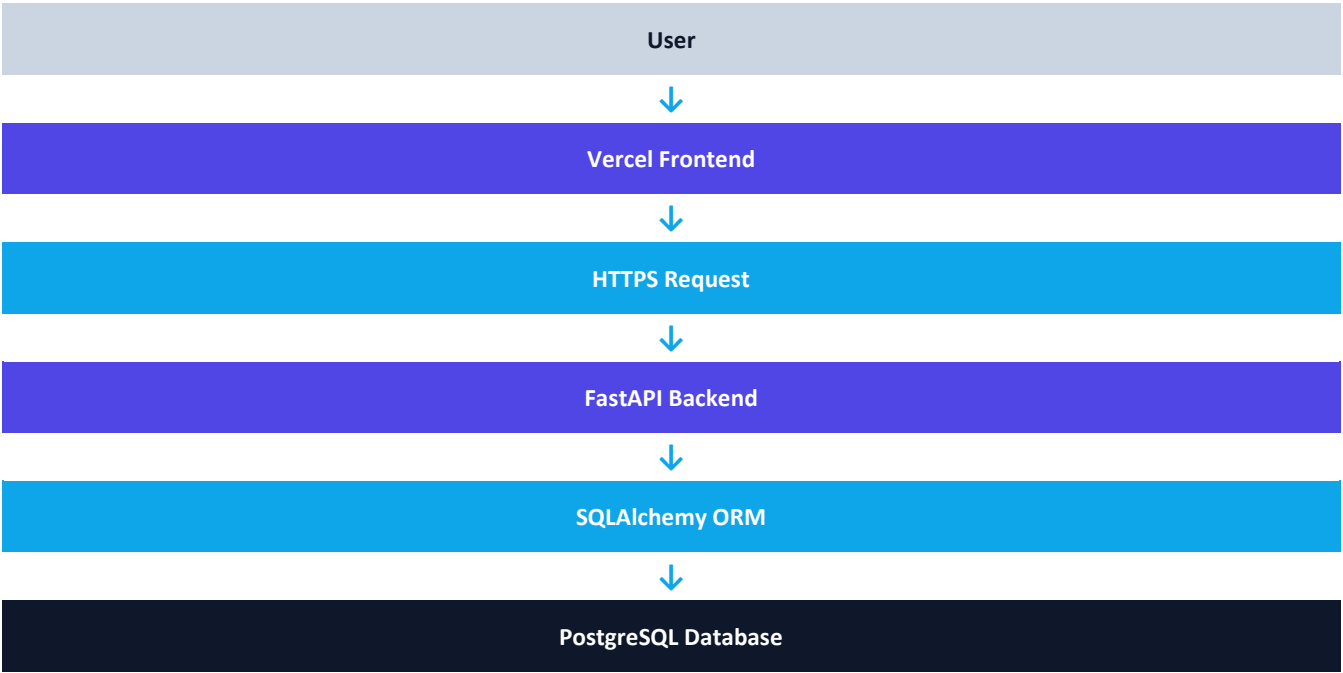
Backend Hosting

- Render
- Railway
- Fly.io

Database Hosting

- Neon PostgreSQL (Recommended)
- Supabase PostgreSQL
- Railway PostgreSQL

Deployment Architecture



PHASE 6 — Future Enhancements	FUTURE ENHANCEMENT
-------------------------------	--------------------

The portfolio will continue to evolve after the initial deployment.

AI Portfolio Assistant

An intelligent chatbot capable of answering questions about:

- Personal Background
- Technical Skills
- Projects
- Internship Experience
- Blogs
- Career Journey

The chatbot will use:

- Retrieval-Augmented Generation (RAG)
- Vector Database (pgvector)
- Large Language Model (OpenAI/Gemini)

It will respond using my communication style and personality while remaining grounded in my actual projects and experiences.

Admin Dashboard

Future administrative features include:

- Secure Login
- Project Management
- Blog Management
- Skill Management
- Certification Management
- Contact Message Management

Blog CMS

Features include:

- Markdown Editor
- Rich Text Editing
- Categories
- Tags
- Draft Support

Analytics Dashboard

Track portfolio usage through:

- Visitor Count
- Popular Projects
- Resume Downloads
- Contact Requests
- Traffic Sources

Authentication

- JWT Authentication
- Role-Based Access Control
- Protected API Endpoints

Performance Optimization

- Lazy Loading

- Image Compression
- Caching
- CDN Integration
- API Rate Limiting

DevOps

Future DevOps improvements include:

- Docker Containerization
- GitHub Actions (CI/CD)
- Automated Testing
- Monitoring
- Health Checks
- Production Logging

5

HIGH-LEVEL ARCHITECTURE

PORTFOLIO WEBSITE

FRONTEND (UI)

HTML • CSS • JavaScript

↓ Fetch API

FASTAPI BACKEND

Routes • Services • ORM

↓ SQLAlchemy

POSTGRESQL DB

Projects • Blogs • Skills • Contacts • Experience • Education

↓ Deployment Layer

Vercel + Render + Neon PostgreSQL

↓ Future Enhancements

AI Assistant • CMS • Analytics • DevOps • CI/CD

6

CURRENT PROJECT STATUS

PHASE	STATUS
Planning & System Design	In Progress
Frontend Development	✓Completed
Database Design	📅 Planned
Backend Development	📅 Planned
API Integration	📅 Planned
Deployment	📅 Planned
AI Assistant	📅 Future Enhancement
Admin Dashboard	📅 Future Enhancement

7

CONCLUSION

This portfolio is designed to evolve from a modern static website into a production-ready full-stack application. The architecture emphasizes scalability, maintainability, and clean software engineering practices by separating the presentation layer, business logic, and data storage.

The immediate next milestone is to design the PostgreSQL database and define API contracts before implementing the FastAPI backend. This contract-first approach will ensure that the frontend and backend remain loosely coupled and easier to maintain. In future iterations, AI capabilities, analytics, an administrative dashboard, and DevOps automation will further transform the portfolio into a comprehensive showcase of both software engineering and AI engineering skills.